



CCT

CENTRO DE CIÊNCIAS TECNOLÓGICAS

Resolução de Problemas com **Grafos**

UNID. 2 Ordenação Topológica, Componentes f-conexas e Fecho transitivo.

Prof. Me. Ricardo Carubbi (carubbi@unifor.br)

1 Grafos Direcionados

1.1 Conceito (Revisão)

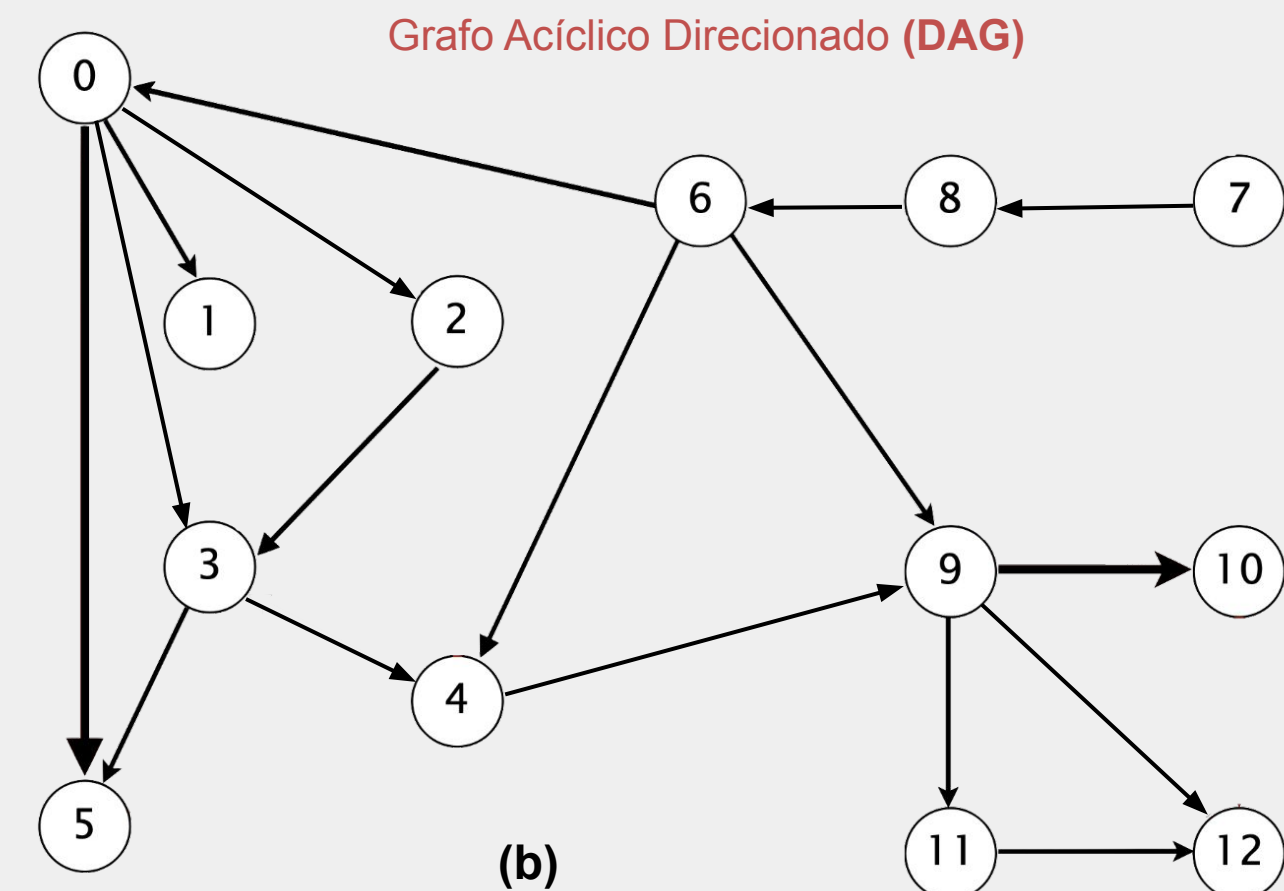
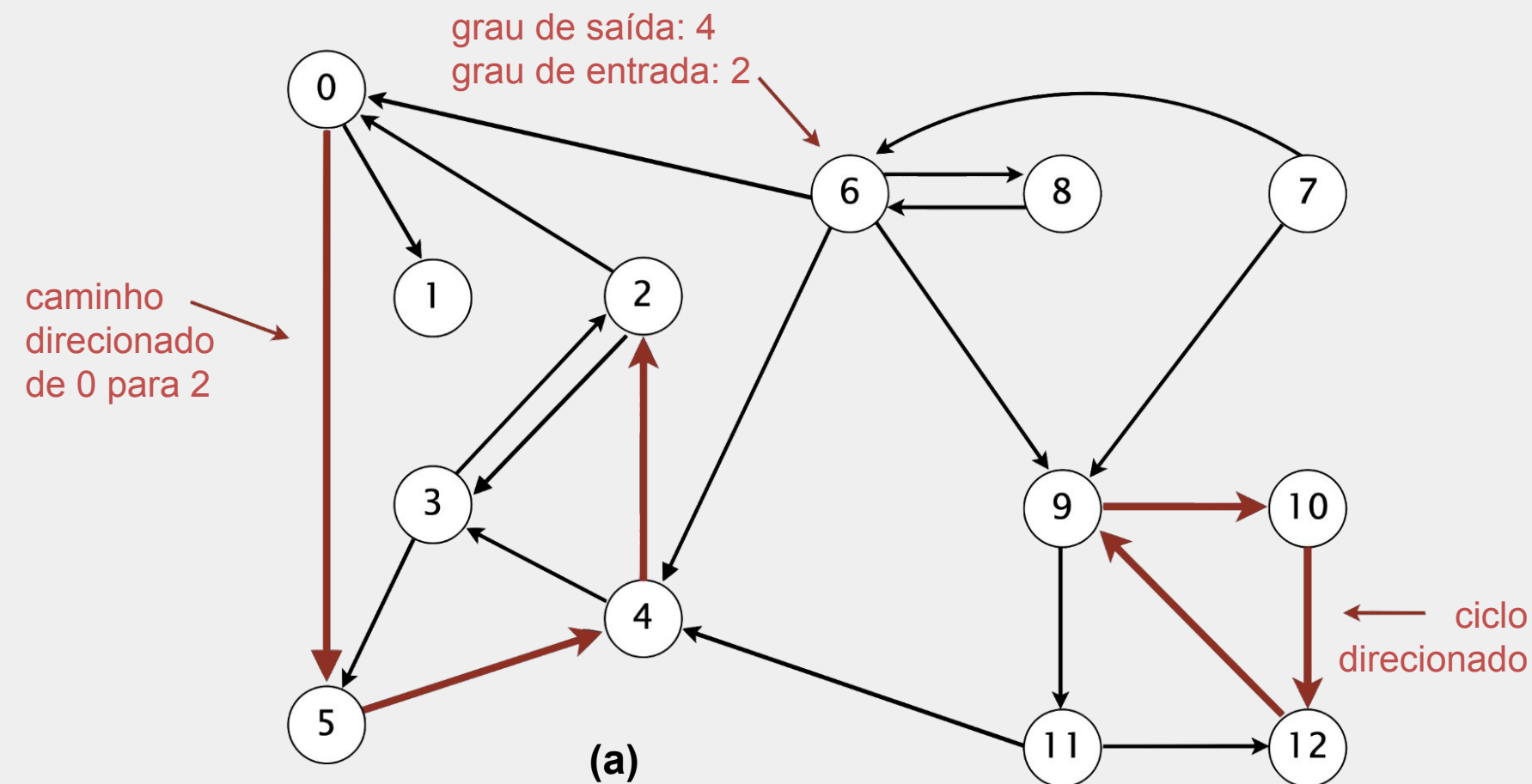


Figura 1 Conceituação de dígrafos: (a) Um dígrafo é definido por uma lista de vértices e arestas, sendo que a ordem em que listamos os vértices ao especificar uma aresta implica que a aresta é dirigida do primeiro vértice para o segundo. (b) Este dígrafo não possui ciclos, uma propriedade que não é imediatamente aparente a partir da lista de arestas ou mesmo ao examinar seu desenho.. Fonte: SEDGEWICK, 2017.

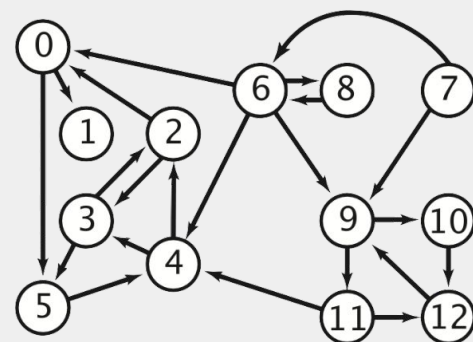
Notas:

1 Grafos Direcionados

1.1 Conceito (Revisão)

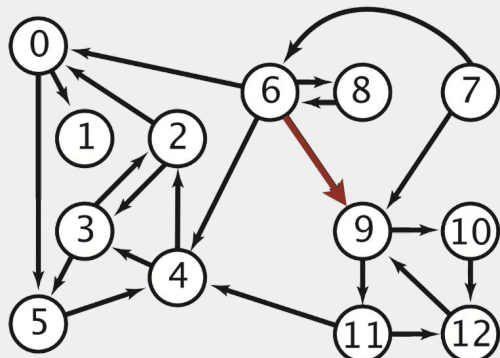
tinyDG.txt

V → 13
22 ← E
4 2
2 3
3 2
6 0
0 1
2 0
11 12
12 9
9 10
9 11
7 9
10 12
11 4
4 3
3 5
6 8
8 6
⋮



(a)

```
% java Digraph tinyDG.txt
0->5
0->1
2->0
2->3
3->5
3->2
4->3
4->2
5->4
⋮
11->4
11->12
12->9
```



(b)

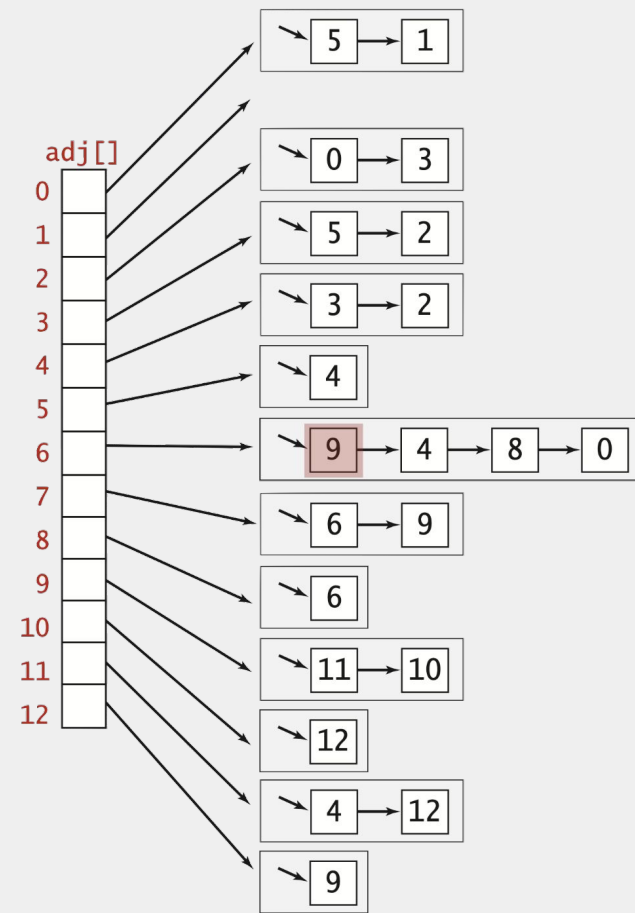
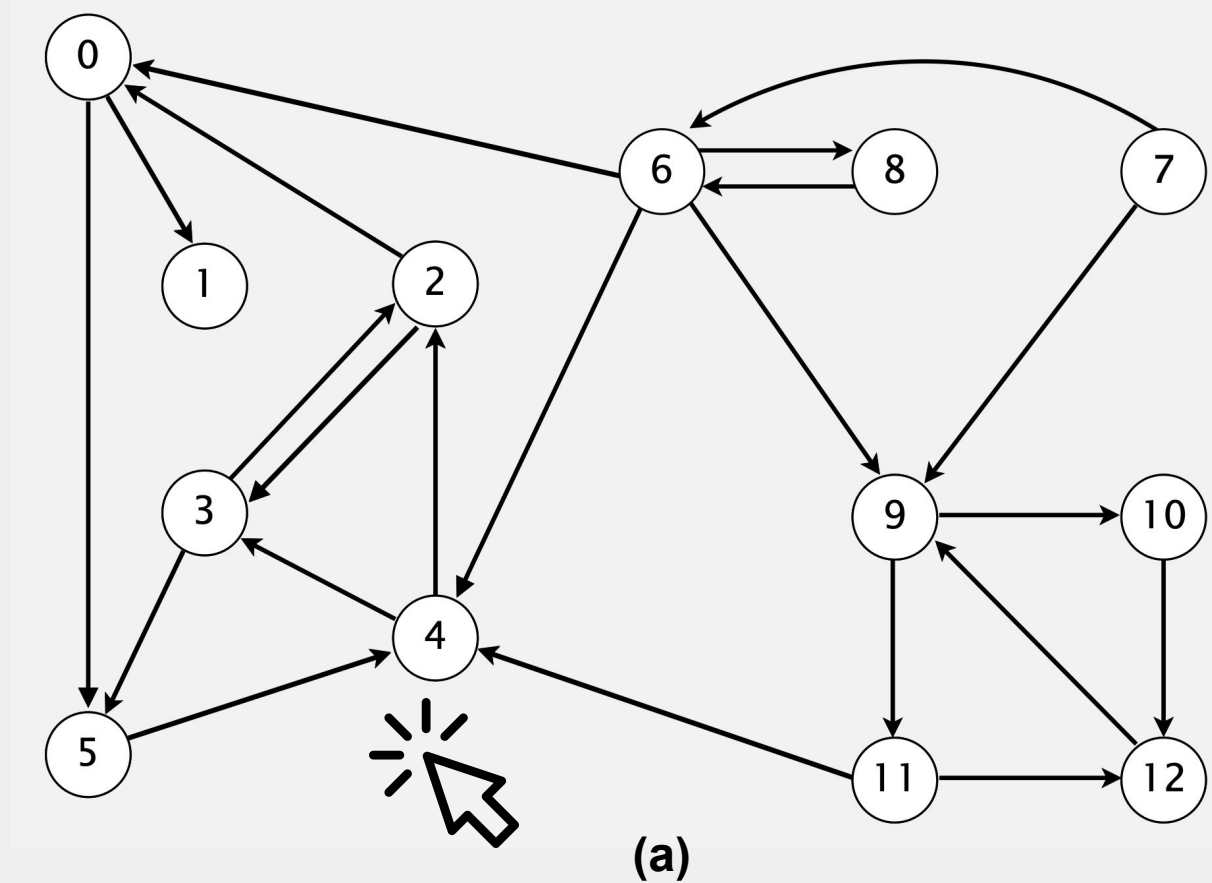


Figura 2 Estrutura de dados num dígrafo: (a) Um dígrafo é definido por uma lista de vértices e arestas, sendo que a ordem em que listamos os vértices ao especificar uma aresta implica que a aresta é dirigida do primeiro vértice para o segundo. (b) Este dígrafo não possui ciclos, uma propriedade que não é imediatamente aparente a partir da lista de arestas ou mesmo ao examinar seu desenho.. Fonte: SEDGEWICK, 2017.

Notas:

1 Grafos Direcionados

1.2 Busca em Profundidade - DFS (Revisão)



tinyG.txt

13
22
4→2
2→3
3→2
6→0
0→1
2→0
11→12
12→9
9→10
9→11
8→9
10→12
11→4
4→3
3→5
6→8
8→6
5→4
0→5
6→4
6→9
7→6

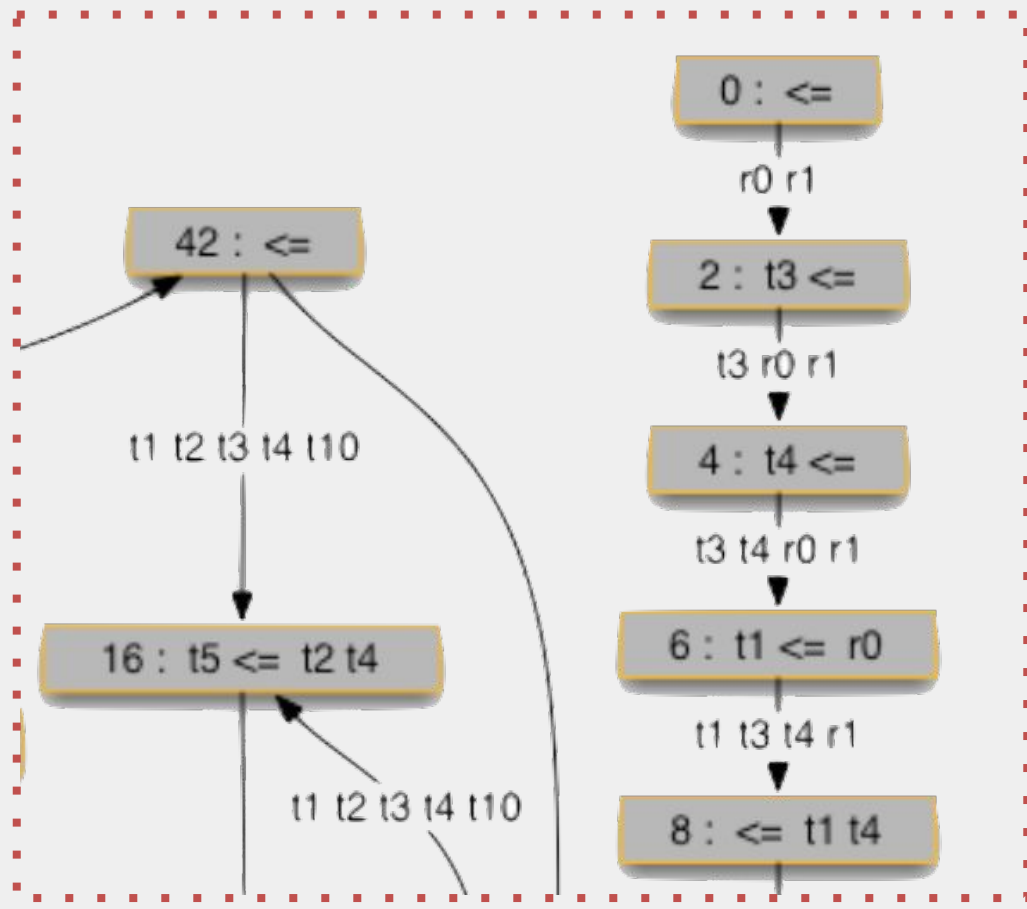
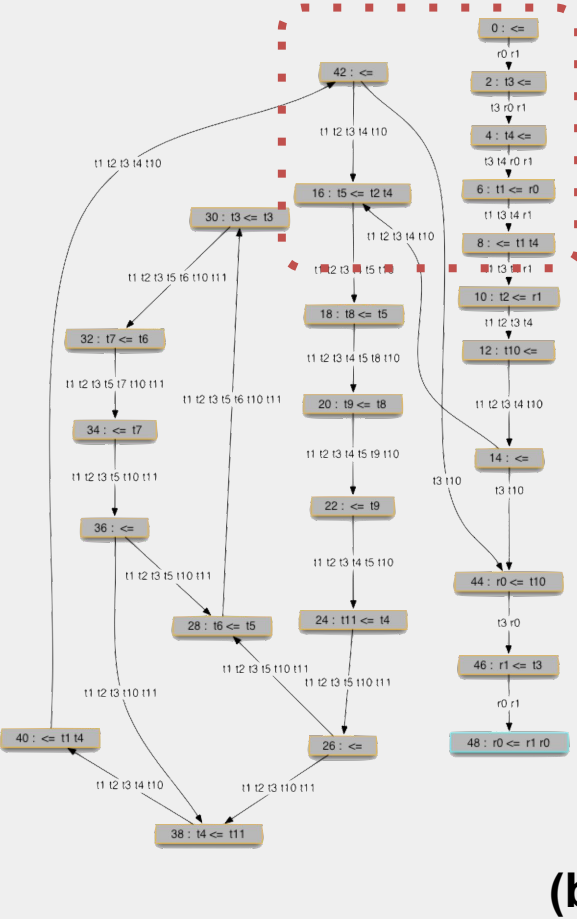


Figura 3 Algoritmos de busca em digrafos. (a) A DFS num grafo direcionado visita recursivamente todos os vértices alcançáveis a partir de um vértice inicial, seguindo apenas o sentido das arestas, até que todos os vértices alcançáveis sejam visitados. (b) Grafo de fluxo de controle de um programa, onde cada vértice representa um bloco básico de instruções e cada aresta representa um salto condicional ou incondicional. Fonte: SEDGEWICK, 2017.

Notas:

1 Grafos Direcionados

1.2 Busca em Largura - BFS (Revisão)

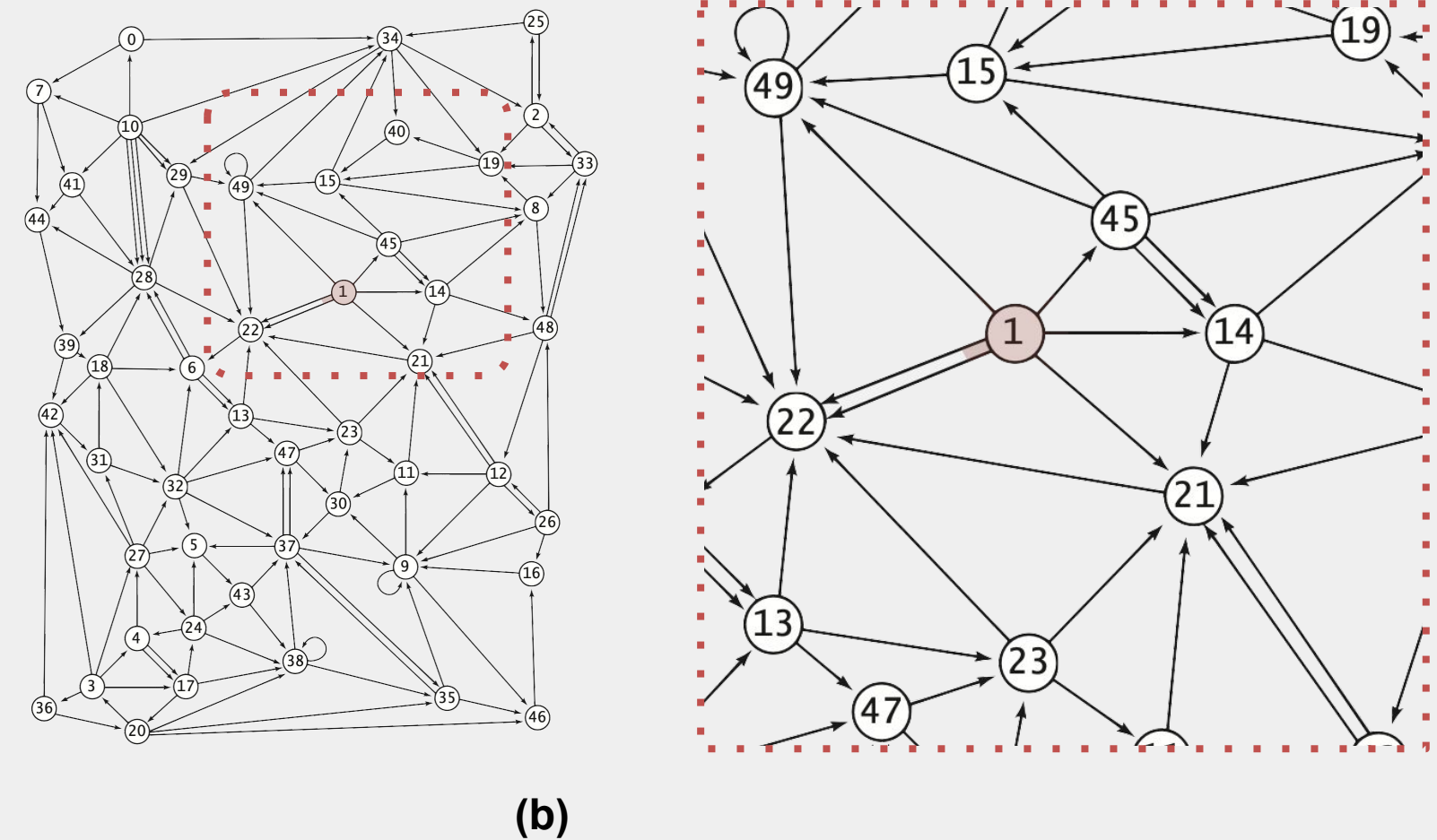
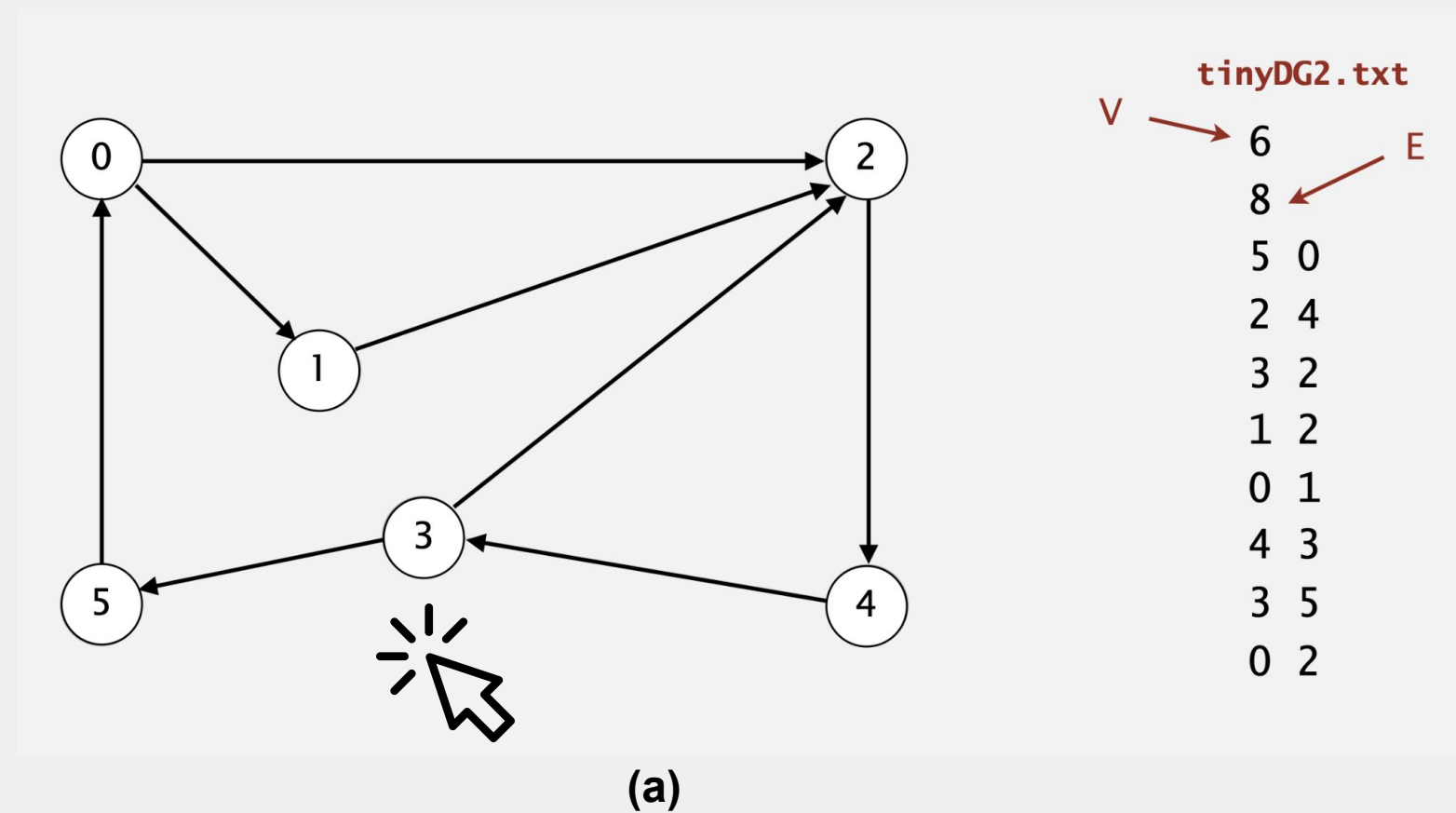


Figura 4 Algoritmos de busca em digrafos. (a) A BFS num digrafo explora todos os vértices alcançáveis a partir de um vértice inicial, visitando primeiro todos os primeiros vizinhos antes de prosseguir na direção dos demais (b) Exploração de páginas web usando BFS: a partir da página raiz, visitamos páginas conectadas por links, mantendo uma fila de exploração e um conjunto de páginas descobertas. Fonte: SEDGEWICK, 2017.

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

1 Grafos Direcionados

1.3 Problemas

Caminho: Existe um caminho direcionado de s para t ?

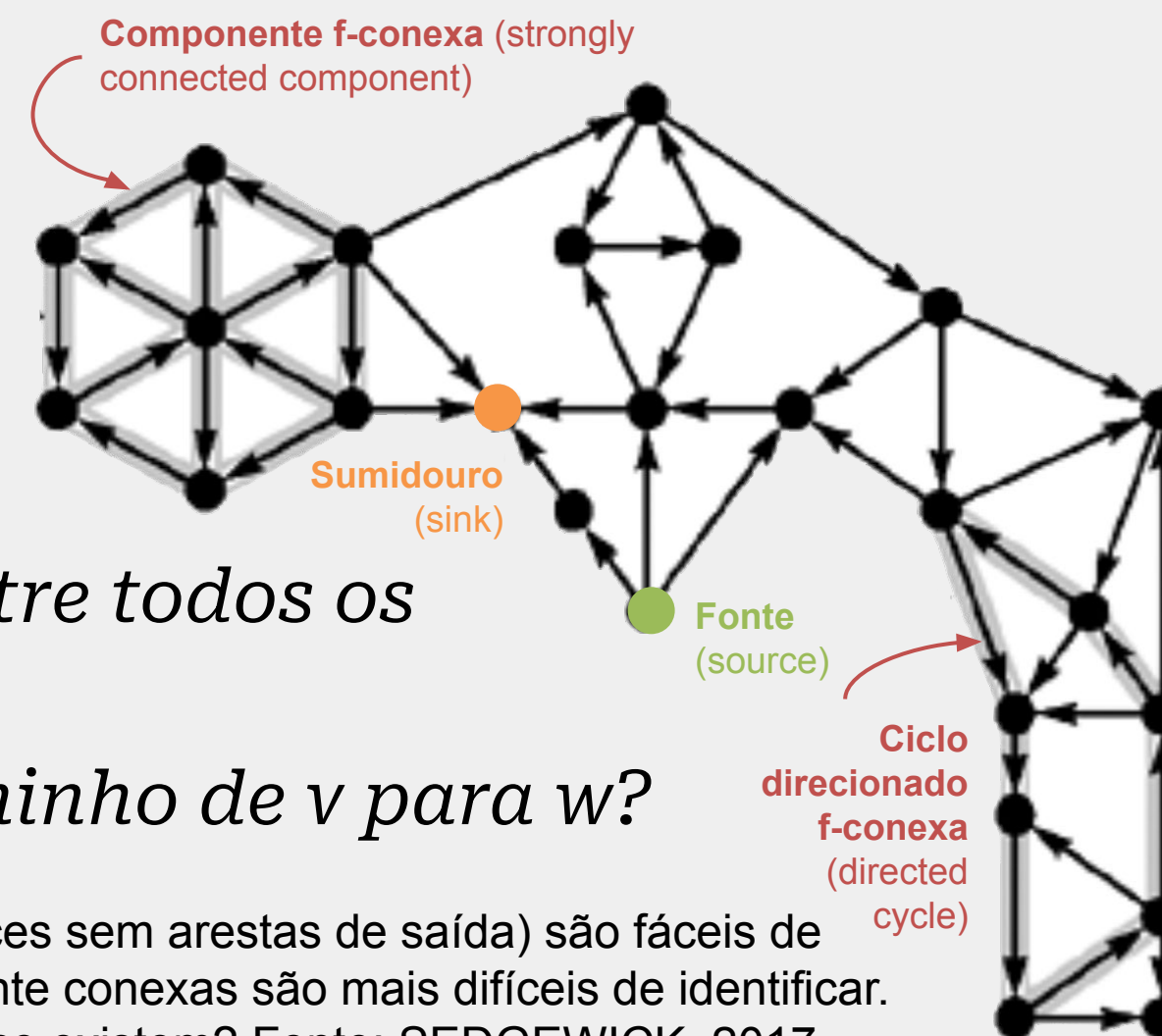
Caminho mais curto: Qual é o caminho direcionado mais curto de s para t ?

Ordenação topológica: Pode-se desenhar um dígrafo de modo que todas as arestas apontem para cima¹?

Conectividade forte: Existe um caminho direcionado entre todos os pares de vértices?

Fecho transitivo: Para quais vértices v e w existe um caminho de v para w ?

Figura 5: Terminologia de dígrafos Fontes (vértices sem arestas de entrada) e sumidouros (vértices sem arestas de saída) são fáceis de identificar em representações de dígrafos como esta, mas ciclos direcionados e componentes fortemente conexas são mais difíceis de identificar. Qual é o maior ciclo direcionado? Quantas componentes fortemente conexas com mais de um vértice existem? Fonte: SEDGEWICK, 2017.



Notas:

1 Significa que cada aresta direcionada parte de um vértice localizado em uma posição inferior e chega a um vértice em uma posição superior, seguindo o sentido vertical ascendente, no caso de dígrafos acíclicos e vértices dispostos de acordo com uma ordenação topológica. Fonte: SEDGEWICK, 2003.

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Ordenação Topológica

2.1 Definição

"Uma ordenação topológica (TS) de um grafo direcionado acíclico (DAG) é uma **ordenação linear** de todos os seus vértices de modo que, para toda aresta dirigida (u,v) , **o vértice u preceda o vértice v na ordenação**"¹.

A TS organiza os vértices de uma DAG de forma a respeitar as **relações de precedência** impostas pelas arestas. O seguinte algoritmo simples ordena topologicamente um DAG:

Topological-Sort(G)²

- (1) chamar $DFS(G)$ para calcular o tempo de término $v.f$ para cada vértice v
- (2) à medida que cada vértice é terminado, inserir o vértice numa pilha
- (3) retorna a pilha inversa de vértices

Notas:

2. Ordenação Topológica

2.1 Definição

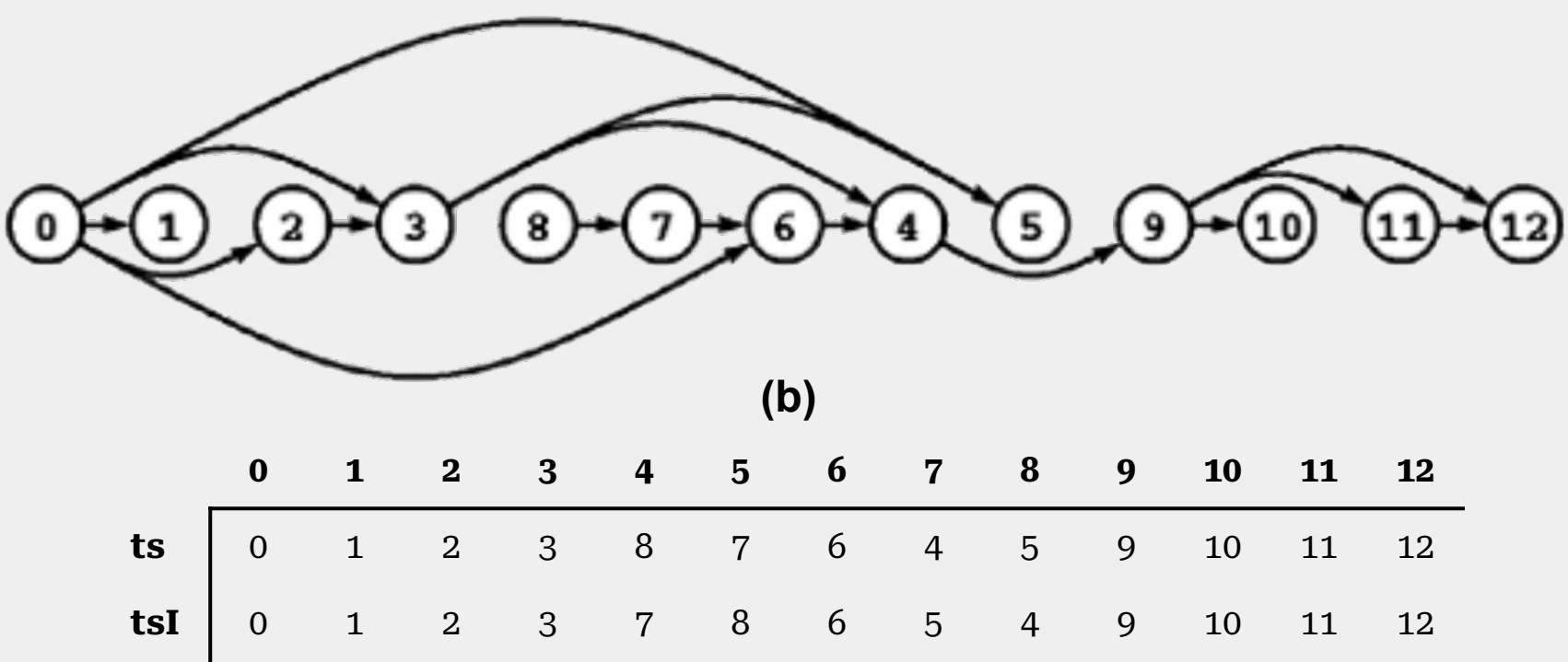
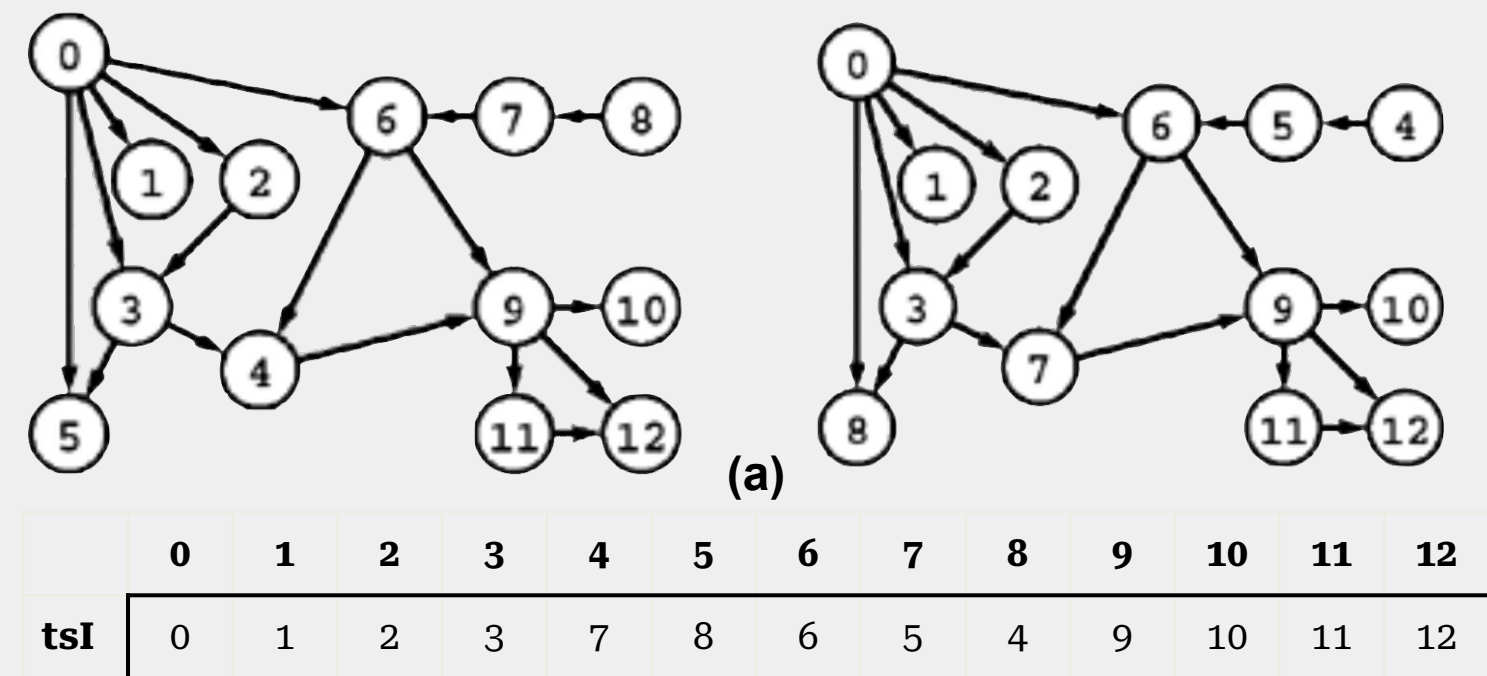


Figura 6 Tipos de ordenação topológica: (a) Dado qualquer DAG (grafo acíclico direcionado) (acima), a ordenação topológica permite que reatribuamos rótulos aos seus vértices de modo que toda aresta aponte de um vértice com número menor para um vértice com número maior (abaixo). Neste exemplo, reatribuímos os rótulos 4, 5, 7 e 8 para 7, 8, 5 e 4, respectivamente, conforme indicado no vetor tsI. (b) Este diagrama mostra outra forma de visualizar a ordenação topológica em (a), em que especificamos uma maneira de reorganizar os vértices, em vez de reatribuí-los. Quando colocamos os vértices na ordem especificada no vetor ts, da esquerda para a direita, todas as arestas direcionadas apontam da esquerda para a direita. A inversa da permutação ts é a permutação tsI, que especifica a reatribuição de rótulos descrita em (a). Fonte: SEDGEWICK, 2017.

Notas:

2. Ordenação Topológica

2.2 Exemplo 1

Objetivo: Dado um conjunto de tarefas a serem concluídas com restrições de precedência, em que ordem devemos agendar as tarefas?

Modelo: Dígrafo Acíclico (DAG) onde vértices são as tarefas e arestas são as relações de precedência.

Tarefas: (0) Algoritmos; (1) Teoria da Complexidade; (2) Inteligência Artificial; (3) Introdução à Ciência da Computação; (4) Criptografia; (5) Computação Científica; (6) Programação Avançada.

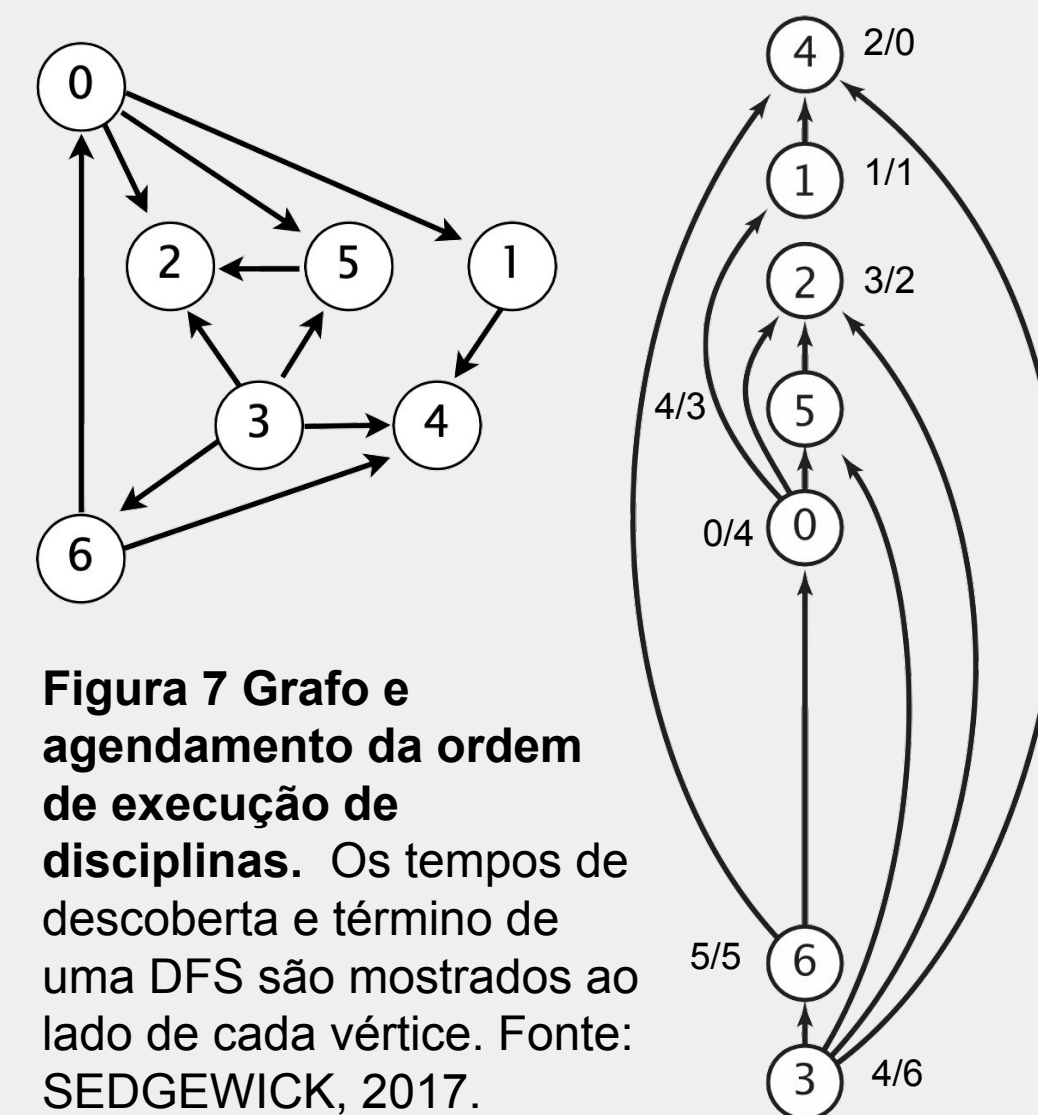


Figura 7 Grafo e agendamento da ordem de execução de disciplinas. Os tempos de descoberta e término de uma DFS são mostrados ao lado de cada vértice. Fonte: SEDGEWICK, 2017.

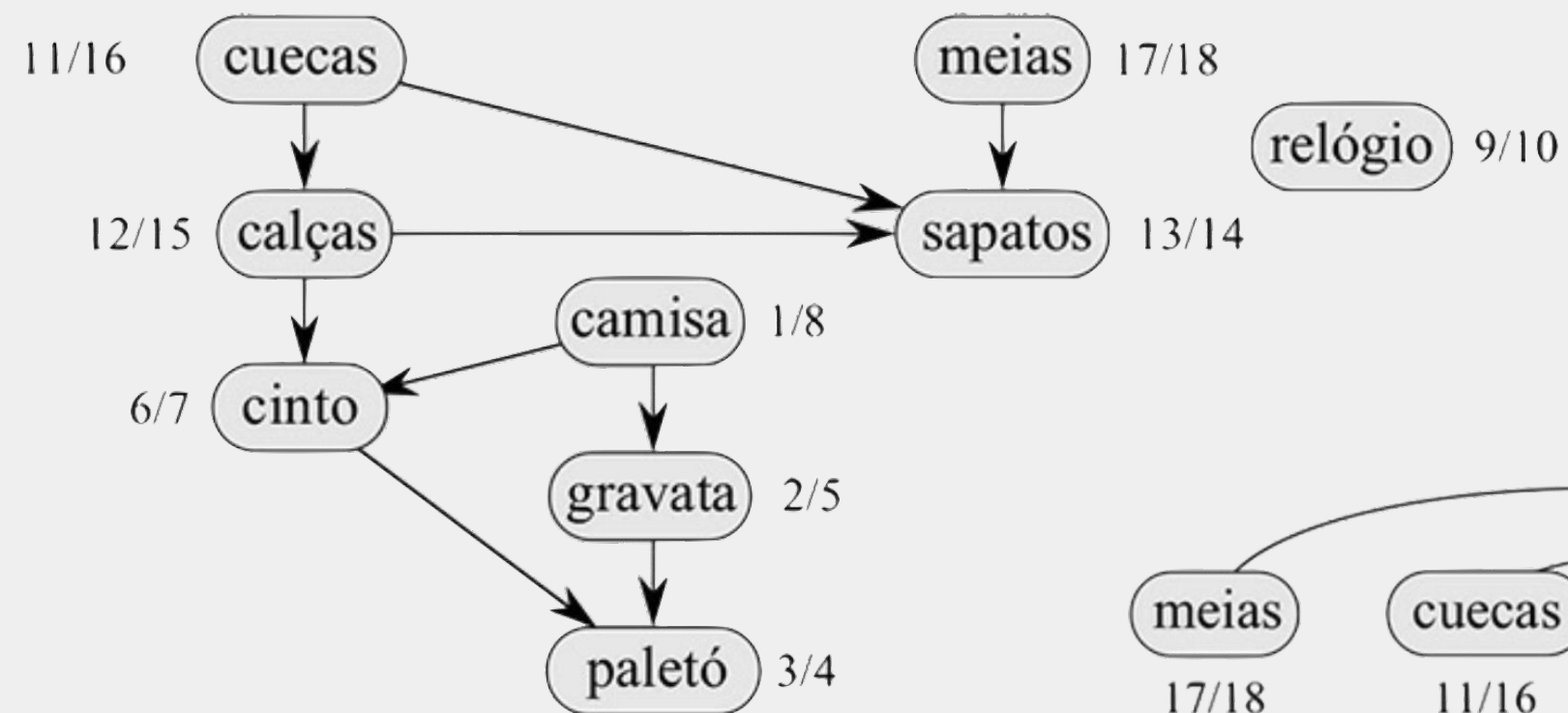
Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Ordenação Topológica

2.2 Exemplo 2



Outro exemplo que surge quando o professor Bumstead se veste pela manhã¹.

O professor deve vestir certas peças de roupa antes de outras (por exemplo, meias antes de sapatos). Outros itens podem ser colocados em qualquer ordem (por exemplo, meias e calças)¹.

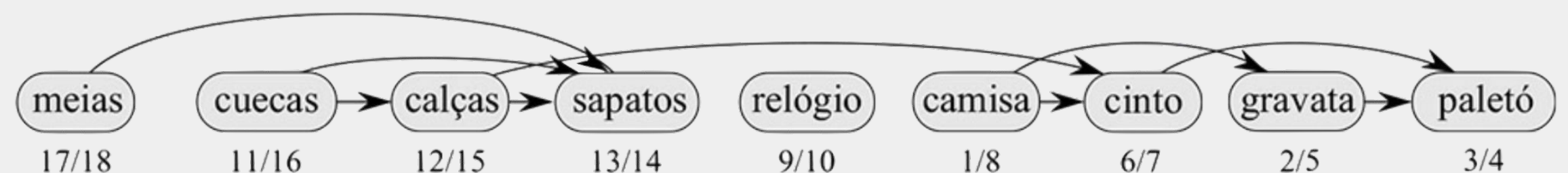


Figura 8 Grafo e ordem para o processo de se vestir. (a) O professor Bumstead ordena topologicamente sua roupa ao se vestir. Cada aresta dirigida (u, v) significa que a peça de roupa u deve ser vestida antes da peça v . Os tempos de descoberta e término de uma busca em profundidade são mostrados ao lado de cada vértice. (b) O mesmo grafo mostrado com uma ordenação topológica. Seus vértices estão organizados da esquerda para a direita, em ordem inversa ou decrescente de tempo de término. Observe que todas as arestas dirigidas vão da esquerda para a direita. Fonte: CORMEM, 2012.

Notas:

2. Ordenação Topológica

2.3 Ordem de busca da DFS

Pré-visita (preorder): Ordem em que os vértices são descobertos (visitados pela primeira vez);

Pós-visita (postorder): Ordem em que os vértices são finalizados (quando todos os seus vizinhos já foram visitados);

Ordem topológica: Ordem inversa dos seus tempos de pós-visita (postorder).

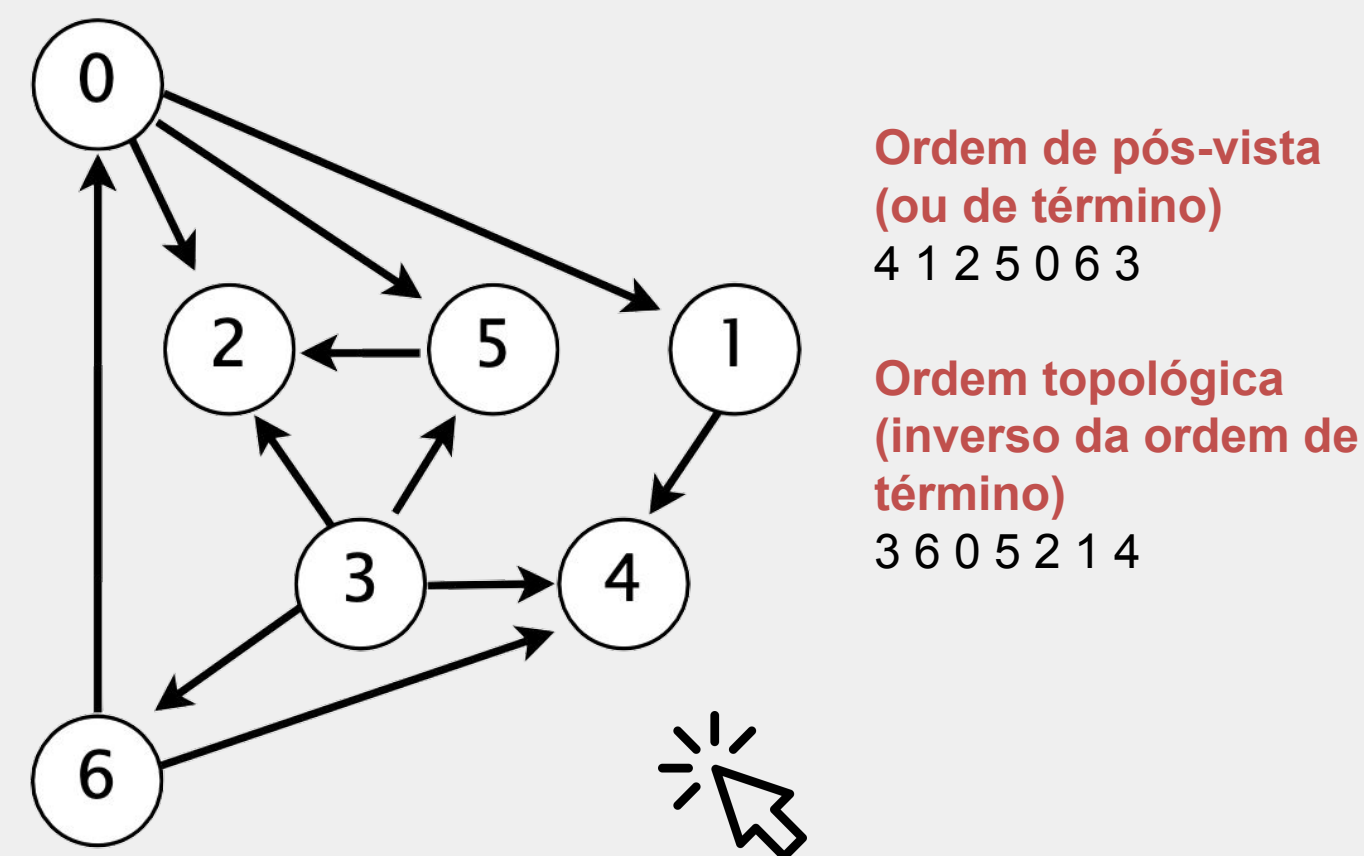


Figura 9 Conceituação de dígrafos: (a) Um dígrafo é definido por uma lista de vértices e arestas, sendo que a ordem em que listamos os vértices ao especificar uma aresta implica que a aresta é dirigida do primeiro vértice para o segundo. (b) Este dígrafo não possui ciclos, uma propriedade que não é imediatamente aparente a partir da lista de arestas ou mesmo ao examinar seu desenho.. Fonte: SEDGEWICK, 2017.

Notas:

2. Ordenação Topológica

2.4 Algoritmo DepthFirstOrder

```
public class DepthFirstOrder {
    private boolean[] marked;           // marked[v] = has v been marked in dfs?
    private int[] pre;                  // pre[v] = preorder number of v
    private int[] post;                 // post[v] = postorder number of v
    private Queue<Integer> preorder;    // vertices in preorder
    private Queue<Integer> postorder;   // vertices in postorder
    private int preCounter;             // counter for preorder numbering
    private int postCounter;            // counter for postorder numbering

    /**
     * Determines a depth-first order for the digraph {@code G}.
     * @param G the digraph
     */
    public DepthFirstOrder(Digraph G) {
        pre = new int[G.V()];
        post = new int[G.V()];
        postorder = new Queue<Integer>();
        preorder = new Queue<Integer>();
        marked = new boolean[G.V()];
        for (int v = 0; v < G.V(); v++)
            if (!marked[v]) dfs(G, v);

        assert check();
    }
}
```

```
// run DFS in digraph G from vertex v and compute preorder/postorder
private void dfs(Digraph G, int v) {
    marked[v] = true;
    pre[v] = preCounter++;
    preorder.enqueue(v);
    for (int w : G.adj(v)) {
        if (!marked[w]) {
            dfs(G, w);
        }
    }
    postorder.enqueue(v);
    post[v] = postCounter++;
}

public Iterable<Integer> reversePost() {
    Stack<Integer> reverse = new Stack<Integer>();
    for (int v : postorder)
        reverse.push(v);
    return reverse;
}
```

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. GOLDBARG, Marco; GOLDBARG, Elizabeth. Grafos: conceitos, algoritmos e aplicações. Elsevier, 2012.

2. Ordenação Topológica

2.5 Algoritmo Topological



```
public class Topological {
    private Iterable<Integer> order; // topological order
    private int[] rank;              // rank[v] = rank of vertex v in order

    /**
     * Determines whether the digraph {@code G} has a topological order and, if so,
     * finds such a topological order.
     * @param G the digraph
     */
    public Topological(Digraph G) {
        DirectedCycle finder = new DirectedCycle(G);
        if (!finder.hasCycle()) {
            DepthFirstOrder dfs = new DepthFirstOrder(G);
            order = dfs.reversePost();
            rank = new int[G.V()];
            int i = 0;
            for (int v : order)
                rank[v] = i++;
        }
    }
}
```

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. GOLDBARG, Marco; GOLDBARG, Elizabeth. Grafos: conceitos, algoritmos e aplicações. Elsevier, 2012.

2. Ordenação Topológica

2.4 Prova da Corretude

Prop. A pós-ordem reversa da DFS em um DAG é uma ordenação topológica.

Prova. Considere qualquer aresta (v,w) . Quando a $dfs(v)$ é chamada:

- **Caso 1:** $dfs(w)$ já foi chamada e retornada. Assim w foi finalizado antes de v .
- **Caso 2:** $dfs(w)$ ainda não foi chamada. $dfs(w)$ será chamada direta ou indiretamente por $dfs(v)$ e terminará antes de $dfs(v)$. Assim, w será finalizada antes de v .
- **Caso 3:** $dfs(w)$ já foi chamada, mas ainda não retornou. Não pode acontecer em um DAG: a pilha de chamadas da função conteria um caminho de w para v , portanto aresta (v,w) completaria um ciclo.

todos os vértices apontados a partir de 3 são finalizados antes de 3 ser finalizado, portanto, eles aparecem depois de 3 na ordenação topológica

```

dfs(0)
  dfs(1)
    dfs(4)
      4 done
    1 done
  dfs(2)
    2 done
  dfs(5)
    check 2
    5 done
  0 done
  check 1
  check 2
  v = 3 → dfs(3)
    caso 1 → check 2
    caso 1 → check 4
    caso 1 → check 5
    caso 2 → dfs(6)
      check 0
      check 4
      6 done
    3 done
    check 4
    check 5
    check 6
    done
  
```

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Ordenação Topológica

2.5 Detecção de Ciclos

Proposição. Um dígrafo possui uma ordenação topológica se e somente se não tiver ciclo direcionado.

Prova.

- Se houver um ciclo direcionado, a ordenação topológica é impossível.
- Se não houver ciclo direcionado, o algoritmo baseado em DFS encontra uma ordenação topológica.

Objetivo. Dado um dígrafo, encontrar um ciclo direcionado.

Solução. DFS com controle de tempos de pré e pós-visita.

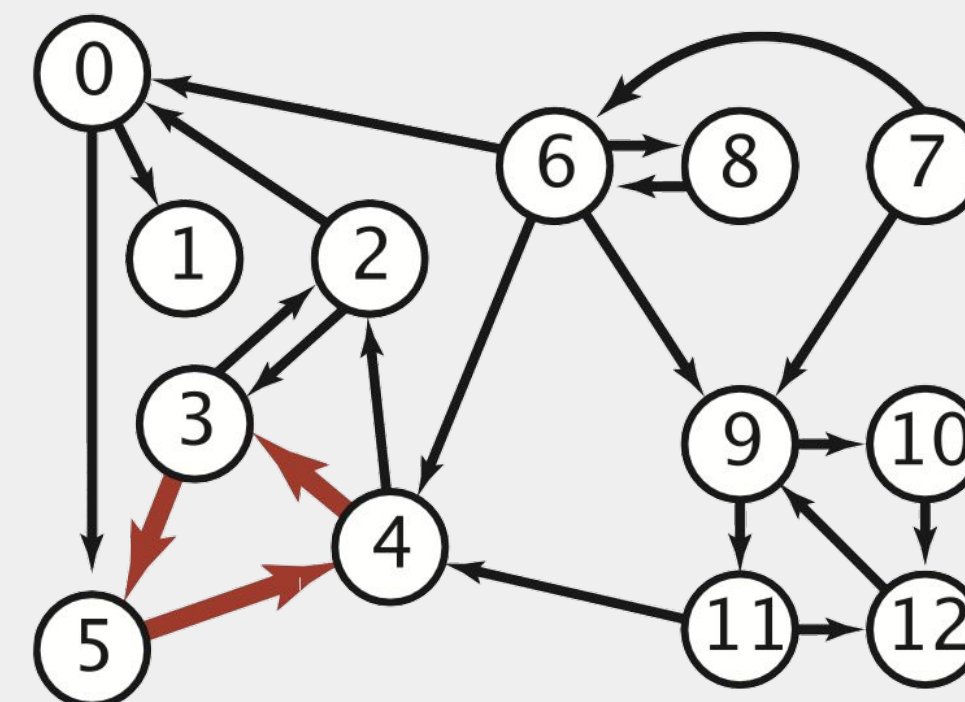


Figura 10: Dígrafo com ciclo direcionado 3-5-4-3. Fonte: SEDGEWICK, 2017.

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

2. Ordenação Topológica

2.6 Classificação dos arcos

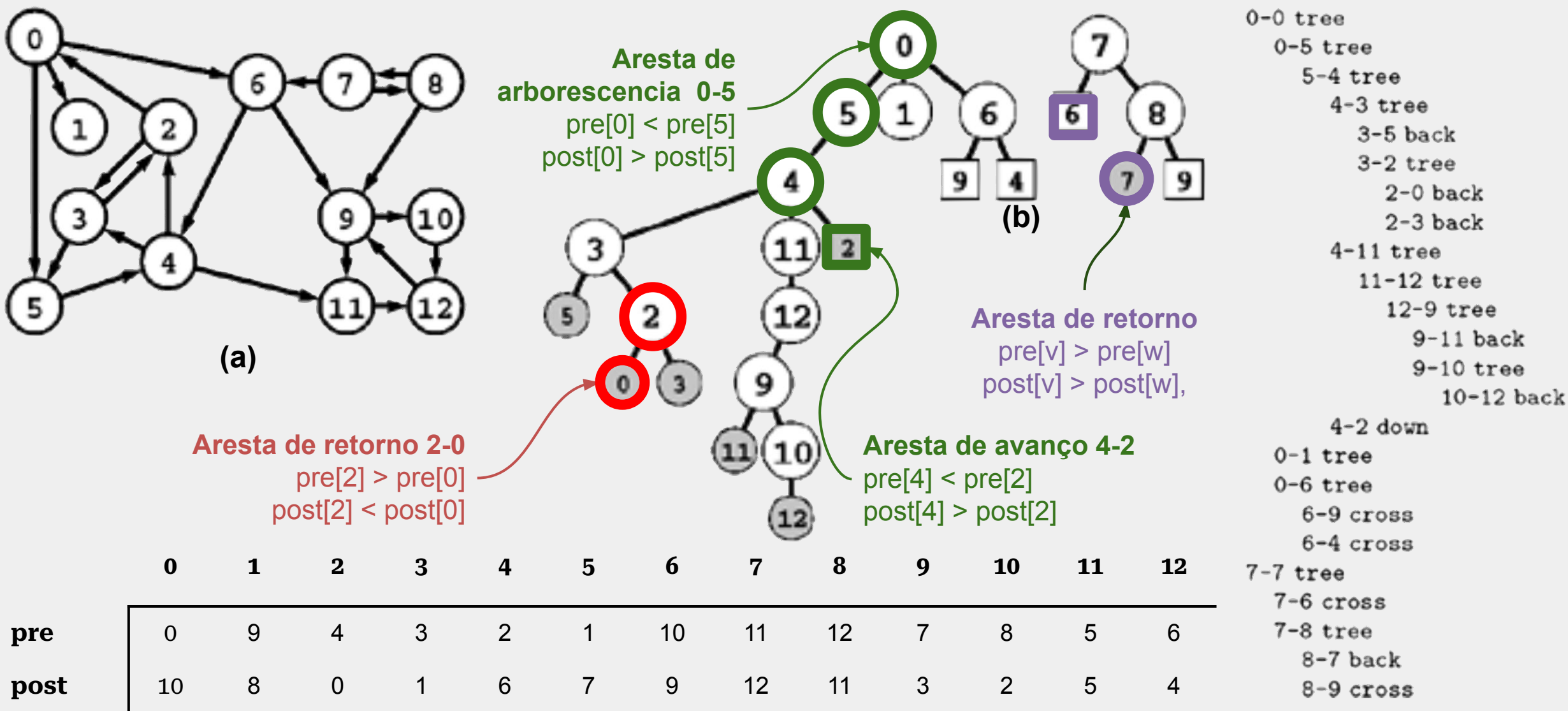
Quatro tipos de arestas derivados de uma DFS:

Tipo de Aresta	Descrição	Condição
Arestas de Arborecência (Tree edge)	Aresta direcionada $v \rightarrow w$ tal que o vértice w é descoberto pela primeira vez durante a chamada recursiva da DFS a partir de v .	w ainda não foi visitado no momento da visita a v .
Arestas de retorno (Back edge)	Aresta $v \rightarrow w$ que conecta um vértice v ao seu ancestral w na árvore de DFS, ou seja, retorna a um vértice já descoberto cuja exploração ainda não terminou. Indicativo de um ciclo.	$pre[v] > pre[w]$ $post[v] < post[w]$
Arestas de avanço (Down edge)	Aresta $v \rightarrow w$ que conecta v a um descendente já totalmente explorado na árvore de DFS, descoberto anteriormente pela mesma DFS.	$pre[v] < pre[w]$ $post[v] > post[w]$
Arestas cruzadas (Cross edge)	Aresta $v \rightarrow w$ que conecta vértices pertencentes a subárvores diferentes da floresta de DFS, isto é, w não é ancestral nem descendente de v .	$pre[v] > pre[w]$ $post[v] > post[w]$

Notas:

2. Ordenação Topológica

2.6 Classificação das arestas



Notas:

Figura 11 Tipos de arestas: A floresta (b) descreve uma busca em profundidade padrão com listas de adjacência no dígrafo da Figura (a). Os nós externos representam vértices já visitados (com o mesmo rótulo do vértice interno correspondente). A floresta representa o dígrafo, com todas as arestas apontando para baixo. Existem quatro tipos de arestas: Arestas de árvore: vão para nós internos. Arestas de retorno: vão para nós externos que representam ancestrais (círculos sombreados). Arestas de avanço: vão para nós externos que representam descendentes (quadrados sombreados). Arestas cruzadas: vão para nós externos que representam vértices que não são ancestrais nem descendentes (quadrados brancos).Fonte: SEDGEWICK, 2017.

2. Ordenação Topológica

2.7 Algoritmo DirectedCycle

```

// run DFS in digraph G from vertex v and compute preorder/postorder
private void dfs(Digraph G, int v) {
    marked[v] = true;
    pre[v] = preCounter++;
    preorder.enqueue(v);
    for (int w : G.adj(v)) {
        if (!marked[w]) {
            dfs(G, w);
        }
    }
    postorder.enqueue(v);
    post[v] = postCounter++;
}

public Iterable<Integer> reversePost() {
    Stack<Integer> reverse = new Stack<Integer>();
    for (int v : postorder)
        reverse.push(v);
    return reverse;
}

```

```

// run DFS and find a directed cycle (if one exists)
private void dfs(Digraph G, int v) {
    onStack[v] = true;
    marked[v] = true;
    for (int w : G.adj(v)) {

        // short circuit if directed cycle found
        if (cycle != null) return;

        // found new vertex, so recur
        else if (!marked[w]) {
            edgeTo[w] = v;
            dfs(G, w);
        }

        // trace back directed cycle
        else if (onStack[w]) {
            cycle = new Stack<Integer>();
            for (int x = v; x != w; x = edgeTo[x]) {
                cycle.push(x);
            }
            cycle.push(w);
            cycle.push(v);
            assert check();
        }
    }
    onStack[v] = false;
}

```

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Componente f-conexas

3.1 Definição

Uma componente fortemente conexa (SCC), ou f-conexa, de um grafo dirigido $G = (V, E)$ é um conjunto maximal de vértices $C \subseteq V$ tal que, para todo par de vértices u e v em C , temos $u-v$ e $v-u$; isto é, u pode ser alcançado a partir do vértice v e vice-versa.

Uma componente f-conexa é um subconjunto máximo de vértices fortemente conectados.

Propriedade fundamental. *A conexão forte é uma relação de equivalência:*

- *v é fortemente conectado a v (reflexiva);*
- *Se v é fortemente conectado a w , então w é fortemente conectado a v (simetria).*
- *Se v é fortemente conectado a w e w a x , então v é fortemente conectado a x (transitividade).*

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Componente f-conexas

3.1 Definição

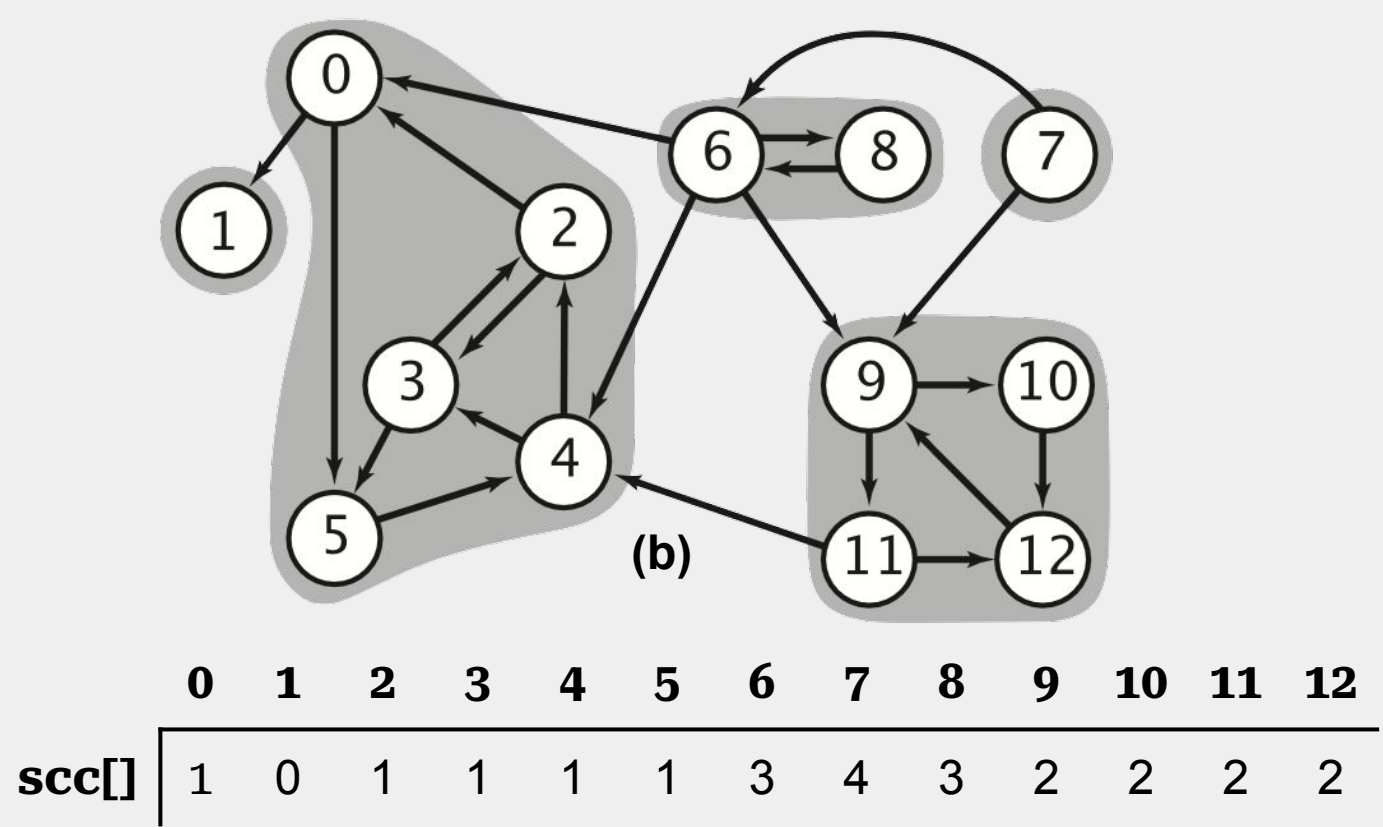
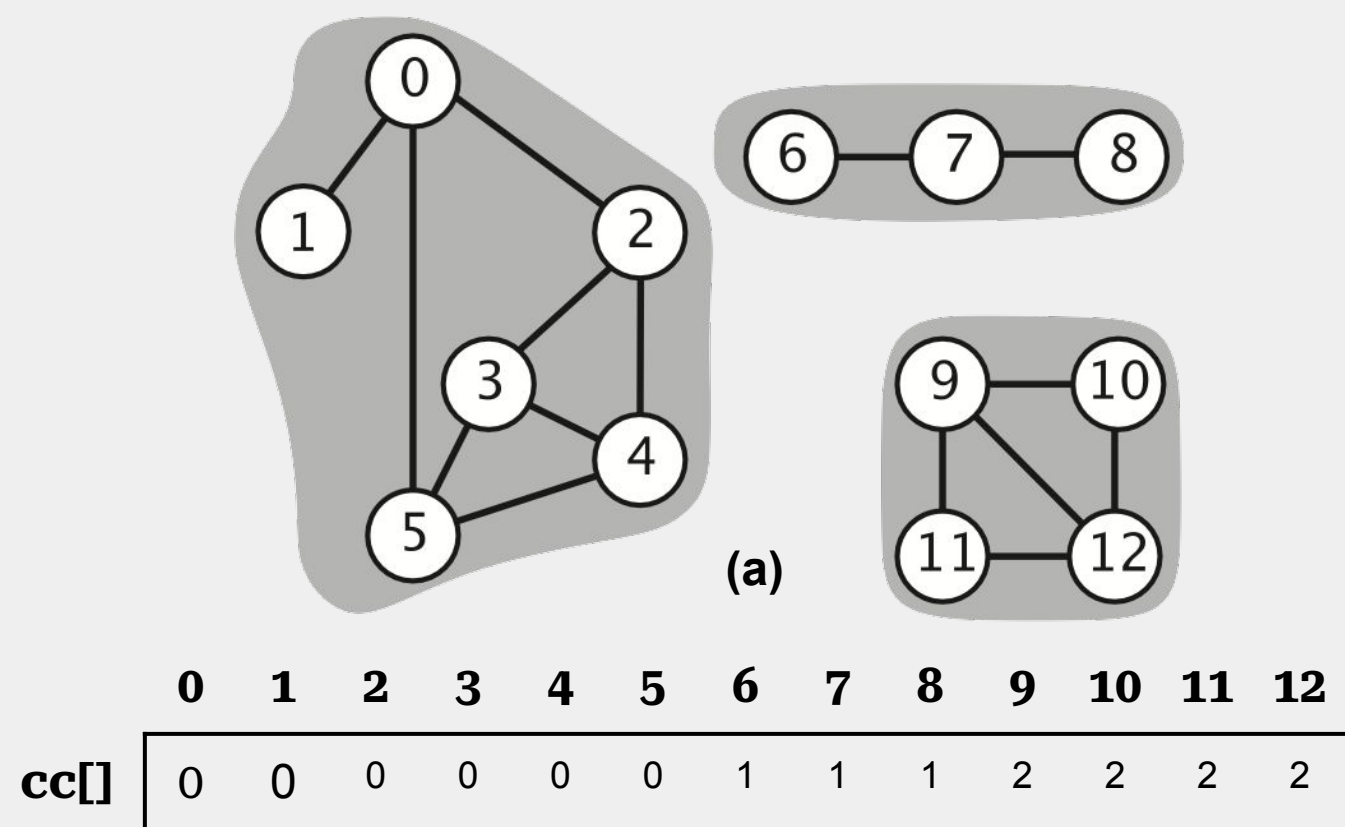


Figura 12 Comparação componentes conexas e componentes f-conexas: (a) v e w estão conectados se existe um caminho entre v e w..(b) v e w são fortemente conectados se existe tanto um caminho dirigido de v para w quanto um caminho dirigido de w para v. Fonte: SEDGEWICK, 2017.

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Componente f-conexas

3.2 Contexto Histórico

Década de 1960: Problema central da Pesquisa Operacional (OR).

- Amplamente estudado; alguns algoritmos práticos.
- Complexidade ainda não compreendida.

Ano 1972: Algoritmo em tempo linear baseado em DFS (Tarjan).

- Algoritmo clássico (nível de dificuldade alto).
- Demonstrou ampla aplicabilidade e importância da DFS.

Década de 1980: Algoritmo simples de duas passagens em tempo linear (Kosaraju-Sharir).

Década de 1990: Novos algoritmos lineares mais simples (Gabow e Gheriyan-Mehlhor).

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Componente f-conexas

3.3 Algoritmo de Kosaraju-Sharir

Grafo reverso: as SCCs em G são as mesmas que em G^T (grafo com todas as arestas invertidas).

Grafo Condensado: Redução de cada componente fortemente conexa em um único vértice.

Ideia:

- Calcular a ordenação topológica (ordem inversa da pós-visita) no DAG núcleo.
- Executar a DFS considerando os vértices na ordem topológica inversa.

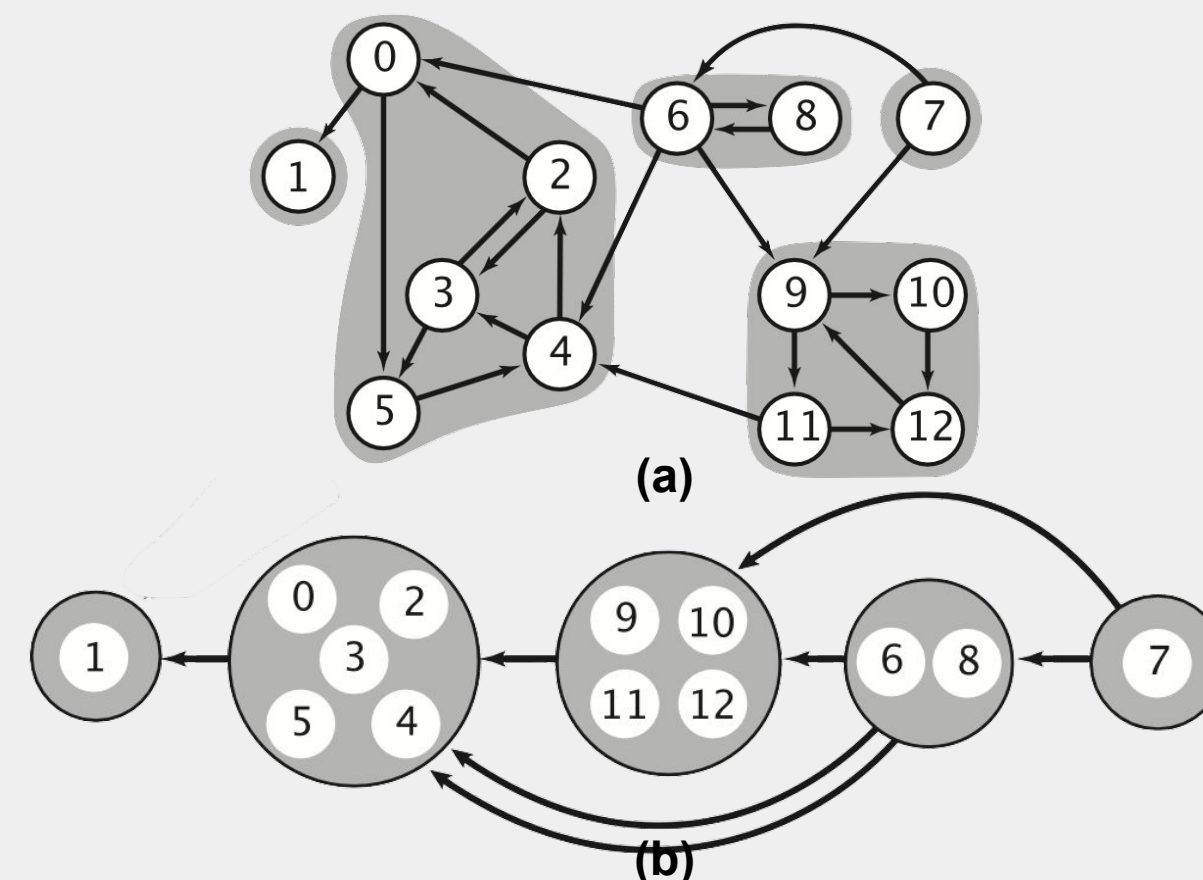


Figura 13 Grafos do algoritmo de Kosaraju-Sharir: (a):Dígrafo original com seus vértices agrupados por componentes fortemente conexas (SCCs). (b) Grafo condensado (DAG) obtido ao contrair cada SCC em um único vértice. As arestas indicam relações de acessibilidade entre os componentes..Fonte: SEDGEWICK, 2017.

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Componente f-conexas

3.3 Algoritmo de Kosaraju-Sharir

Fase 1. Calcular a ordem pós-fixada reversa em G^T (grafo reverso).

Fase 2. Executar DFS em G , visitando os vértices não marcados na ordem pós-fixada reversa de G^T .

Complexidade: $O(V + E)$. Gargalo está em executar a DFS duas vezes (e em calcular G^T).

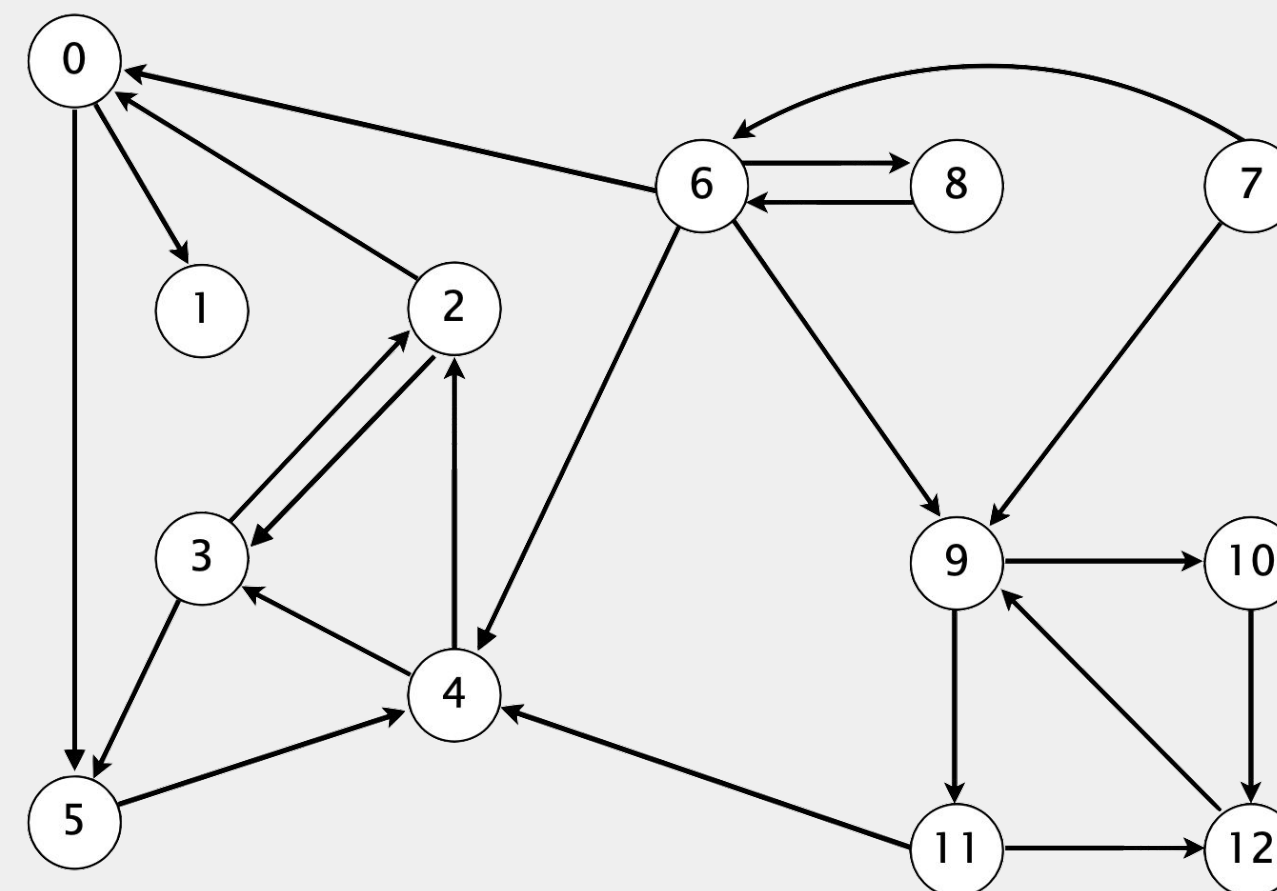


Figura 14 Desenvolvimento do algoritmo de Kosaraju-Sharir: passo a passo como funciona o algoritmo de Kosaraju-Sharir para encontrar componentes fortemente conexas em um dígrafo. Fonte: SEDGEWICK, 2017.

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

3. Componente f-conexas

3.4 Algoritmo KosarajuSharirSCC

```

public class KosarajuSharirSCC {
    private boolean[] marked;    // marked[v] = has vertex v been visited?
    private int[] id;            // id[v] = id of strong component containing v
    private int count;           // number of strongly-connected components

    /**
     * Computes the strong components of the digraph {@code G}.
     * @param G the digraph
     */
    public KosarajuSharirSCC(Digraph G) {

        // compute reverse postorder of reverse graph
        DepthFirstOrder dfs = new DepthFirstOrder(G.reverse());

        // run DFS on G, using reverse postorder to guide calculation
        marked = new boolean[G.V()];
        id = new int[G.V()];
        for (int v : dfs.reversePost()) {
            if (!marked[v]) {
                dfs(G, v);
                count++;
            }
        }

        // check that id[] gives strong components
        assert check(G);
    }

```

```

/**
 * Returns the number of strong components.
 * @return the number of strong components
 */
public int count() {
    return count;
}

public boolean stronglyConnected(int v, int w) {
    validateVertex(v);
    validateVertex(w);
    return id[v] == id[w];
}

public int id(int v) {
    validateVertex(v);
    return id[v];
}

// does the id[] array contain the strongly connected components?
private boolean check(Digraph G) {
    TransitiveClosure tc = new TransitiveClosure(G);
    for (int v = 0; v < G.V(); v++) {
        for (int w = 0; w < G.V(); w++) {
            if (stronglyConnected(v, w) != (tc.reachable(v, w) && tc.reachable(w, v)))
                return false;
        }
    }
    return true;
}

// throw an IllegalArgumentException unless {@code 0 <= v < V}
private void validateVertex(int v) {
    int V = marked.length;
    if (v < 0 || v >= V)
        throw new IllegalArgumentException("vertex " + v + " is not between 0 and " + (V-1));
}

```

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

4. Fecho Transitivo

4.1 Definição

Dado um grafo direcionado G , o fecho transitivo é um grafo que indica, para cada par de vértices v e w , se existe um caminho de v até w em G .

Representação: Matriz booleana onde $M[v][w]$ é 1 se e somente se w é alcançável a partir de v .

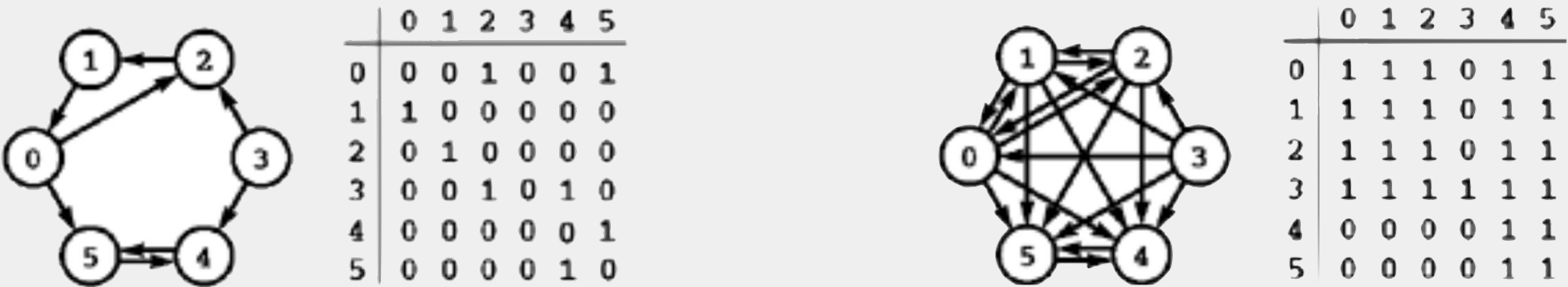


Figura 15 Representação de fecho transitivo: Dígrafo (a) possui apenas oito arestas direcionadas, mas seu fecho transitivo (b) mostra que existem caminhos direcionados conectando 19 dos 30 pares de vértices. Propriedades estruturais do dígrafo são refletidas no fecho transitivo. Por exemplo, as linhas 0, 1 e 2 na matriz de adjacência do fecho transitivo são idênticas, porque esses vértices formam um ciclo direcionado no dígrafo.. Fonte: SEDGEWICK, 2017.

Notas:

4. Fecho Transitivo

4.2 Propriedades

Seja A a matriz de adjacência de um dígrafo G , sendo $C = A \times A = A^2$; conclui-se que:

- Para cada par de vértices s e t , existe uma aresta de s para t em C se, e somente se existe um vértice i tal que haja um caminho de s até i e um caminho de i até t em A .

$$C[s][t] = \bigvee_{i=0}^{V-1} (A[s][i] \wedge A[i][t])$$

V : número de vértices no grafo;

$\wedge$: operador lógico AND (conjunção booleana);

$\vee$: operador lógico OR (disjunção booleana);

$\forall$: disjunção em todos os valores de i .

Para cada i , verificamos se há um caminho de s para t passando por i .

A operação $A[s][i] \wedge A[i][t]$ é 1 se:

- Existe uma aresta de $s \rightarrow i$ e
- Existe uma aresta de $i \rightarrow t$

A disjunção ($\vee$) verifica se pelo menos um desses caminhos indiretos existe.

Notas:

1. A multiplicação booleana de matrizes $A \times A = A^2$ é feita com as operações de soma (OU) e produto (E).

2. Para cada i , verificamos se há um caminho de s para t passando por i . A disjunção ($\vee$) verifica se pelo menos um desses caminhos indiretos existe.

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.

2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

4. Fecho Transitivo

4.2 Propriedades

Seja A a matriz de adjacência de um dígrafo G , sendo $C = A \times A = A^2$; conclui-se que:

- Em outras palavras, as arestas direcionadas em A^2 correspondem exatamente aos caminhos direcionados de comprimento 2 em A .*
- Se incluirmos loops em todos os vértices de A , então A^2 também conterá as arestas de A , caso contrário, não.*

$$A[s][s] \wedge A[s][t] = 1 \wedge 1 = 1 \Rightarrow A^2[s][t] = 1$$

$$A[s][t] = 1 \Rightarrow A^2[s][t] = 1 \quad \text{se e somente se} \quad A[s][s] = 1$$

Notas:

1. A multiplicação booleana de matrizes $A \times A = A^2$ é feita com as operações de soma (OU) e produto (E).

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.

2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

4. Fecho Transitivo

4.2 Propriedades

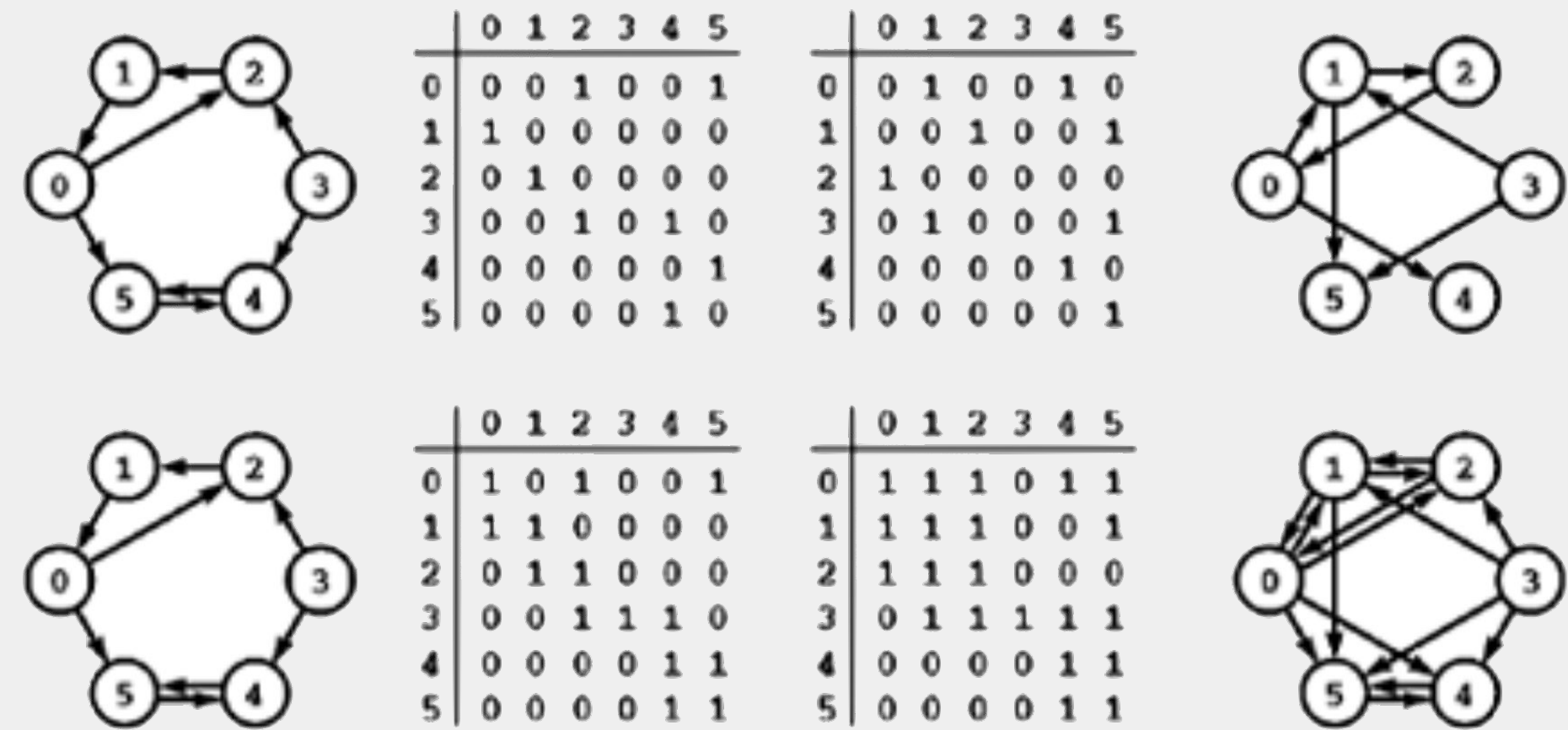


Figura 16 Elevar ao quadrado uma matriz de adjacência: Se colocarmos 0s na diagonal da matriz de adjacência de um dígrafo (a), o quadrado da matriz representa um grafo com uma aresta correspondente a cada caminho de comprimento 2 (b). Se colocarmos 1s na diagonal (c), o quadrado da matriz representa um grafo com uma aresta correspondente a cada caminho de comprimento 1 ou 2 (d). Fonte: SEDGEWICK, 2017.

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

4. Fecho Transitivo

4.2 Propriedades

O fecho transitivo de um dígrafo pode ser calculado construindo sua matriz de adjacência A , adicionando loops em todos os vértices e calculando A^V .

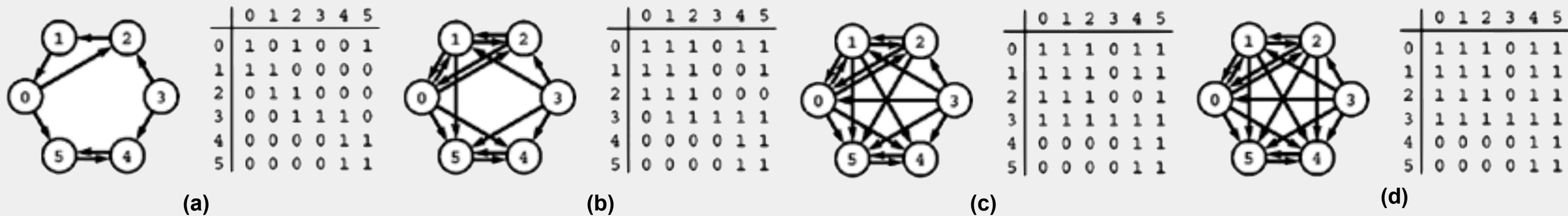


Figura 17 Potências da matriz de adjacência e caminhos direcionados. Mostra as potências da matriz de adjacência, que geram grafos com arestas correspondentes a todos os caminhos de comprimento menor que 1, 2, 3 e 4. O grafo D representa o fecho transitivo deste exemplo, pois não existem caminhos de comprimento maior que 4 que conectem vértices que ainda não tenham sido conectados por caminhos mais curtos. Fonte: SEDGEWICK, 2017.

Notas:

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.
2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.

4. Fecho Transitivo

4.3 Algoritmo TransitiveClosure

```
public class TransitiveClosure {
    private DirectedDFS[] tc; // tc[v] = reachable from v

    public TransitiveClosure(Digraph G) {
        tc = new DirectedDFS[G.V()];
        for (int v = 0; v < G.V(); v++)
            tc[v] = new DirectedDFS(G, v);
    }

    public boolean reachable(int v, int w) {
        validateVertex(v);
        validateVertex(w);
        return tc[v].marked(w);
    }

    private void validateVertex(int v) {
        int V = tc.length;
        if (v < 0 || v >= V)
            throw new IllegalArgumentException("vertex " + v + " is not between 0 and " + (V-1));
    }
}
```

Declara um **vetor de objetos DirectedDFS** chamado **tc**, onde cada posição **tc[v]** representa o conjunto de **vértices alcançáveis a partir de v** por DFS.

Inicializa o vetor **tc** com o tamanho igual ao número de vértices do grafo **G**. Ou seja, haverá uma busca DFS iniciada em cada vértice do grafo.

Método que responde se existe um caminho direcionado de **v** até **w** no grafo.

Consulta a DFS feita a partir de **v** e retorna se o vértice **w** foi alcançado durante essa DFS.

Método auxiliar privado que lança uma exceção se o índice **v** estiver fora dos limites válidos.

Notas:

1. Algoritmo que calcula o fecho transitivo de um dígrafo executando uma DFS a partir de cada vértice e armazenando os resultados. Essa solução é ideal para dígrafos pequenos ou densos, mas não é adequada para dígrafos grandes, pois o construtor utiliza espaço proporcional a V^2 e tempo proporcional a $V(V+E)$.

CENTRO DE CIÊNCIAS TECNOLÓGICAS

1. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: The MIT Press, 2009.

2. SEDGEWICK, Robert. Algorithms in Java, Part 5: Graph Algorithms: Graph Algorithms. Addison-Wesley Professional, 2003.



Obrigado

e até a próxima aula!